入侵检测规则匹配算法--单模匹配算法、多模匹配算法、hyperscan

原创 网络安全研发随想 于 2020-04-10 01:24:28 发布 阅读量1w 收藏 44 点赞数 7 版权
分类专栏： 网络安全

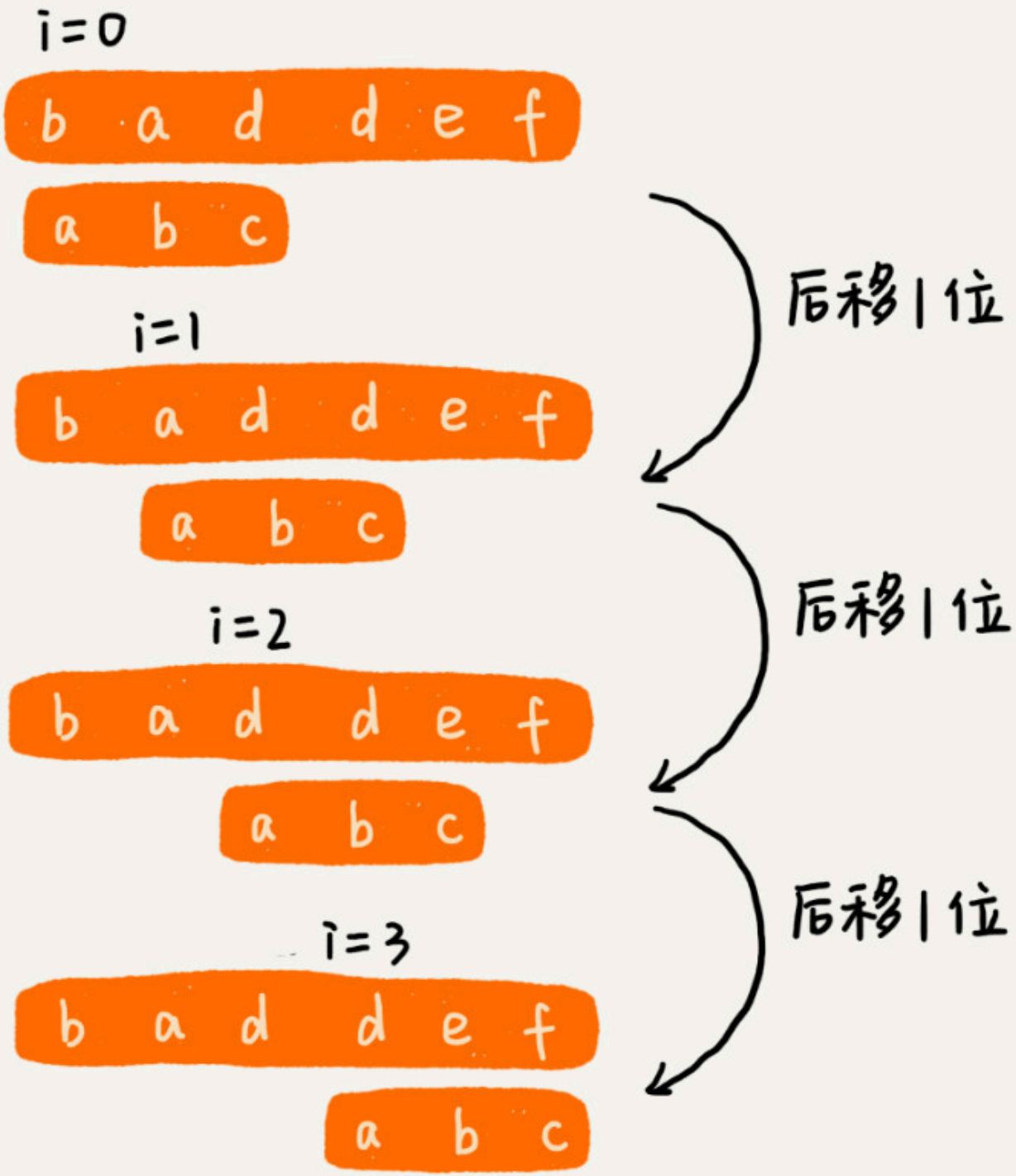
入侵检测规则匹配算法，分为单模式匹配算法和多模式匹配算法。

1. 单模式匹配

单模式匹配，就是一个串跟一个串进行匹配，常见算法有：BM算法和KMP算法。

1.1 BF （Brute Force）暴力匹配算法

作为最简单、最暴力的字符串匹配算法，BF 算法的思想可以用一句话来概括，那就是，我们在主串中，检查起始位置分别是 0、1、2…n-m 且长度为 m 的 n-m+1 个子串，看有没有跟模式串匹配的。



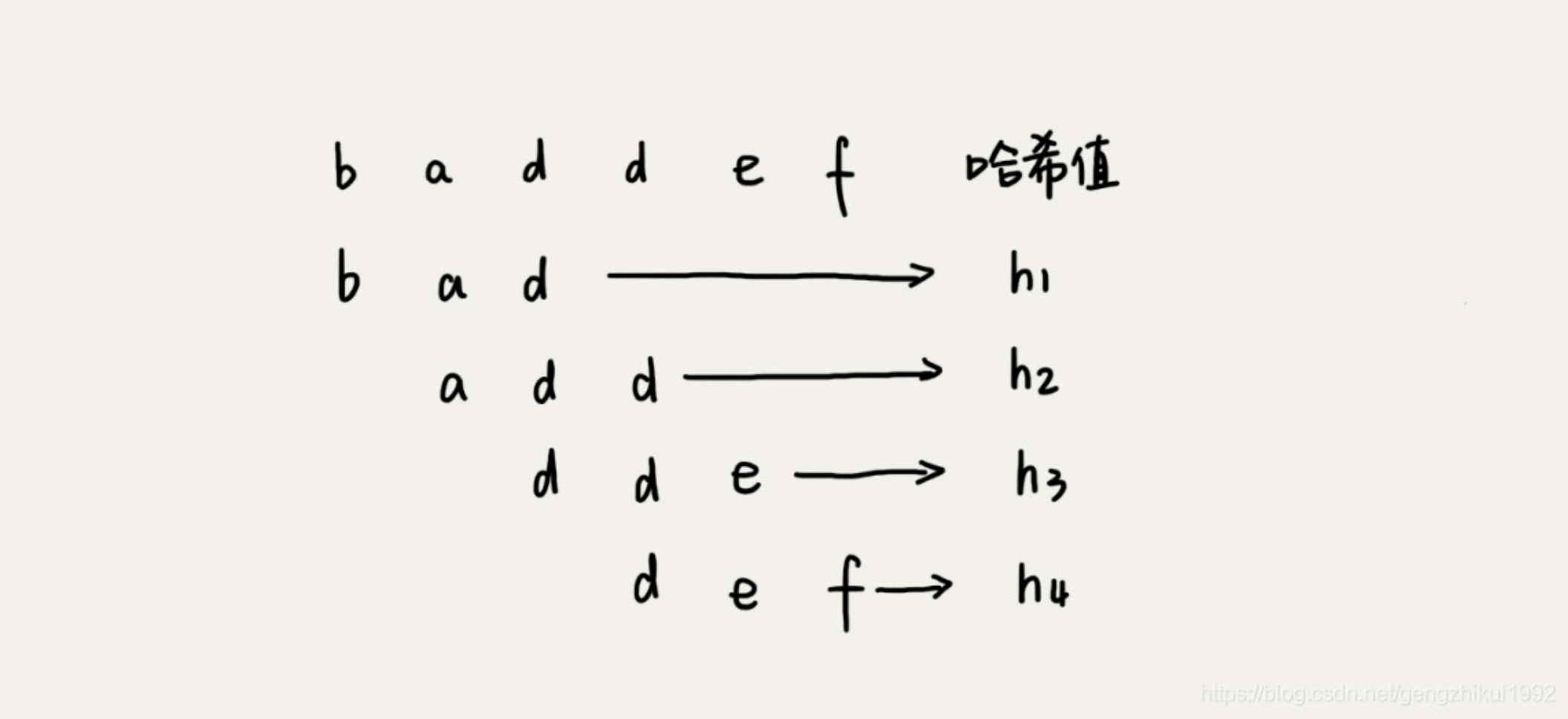
https://blog.csdn.net/gengzhikui1992

- 理论上的最坏情况时间复杂度是 O(n*m)，但是，统计意义上，大部分情况下，算法执行效率要比这个高很多。
- 朴素字符串匹配算法思想简单，代码实现也非常简单。简单意味着不容易出错，如果有 bug 也容易暴露和修复。

2.2 RK（Rabin-Karp） 算法

BF算法每次检查主串与子串是否匹配，需要依次比对每个字符，所以 BF 算法的时间复杂度就比较高，是 O(n*m)。

RK 算法的思路是这样的：
我们通过哈希算法对主串中的 n-m+1 个子串分别求哈希值，然后逐个与模式串的哈希值比较大小。如果某个子串的哈希值与模式串相等，那就说明对应的子串和模式串匹配了。



模式串哈希值与每个子串哈希值之间的比较的时间复杂度是 O(1)，总共需要比较 n-m+1 个子串的哈希值，所以，这部分的时间复杂度也是 O(n)。所以，RK 算法整体的时间复杂度就是 O(n)。

跟 BF 算法相比，效率提高了很多。不过这样的效率取决于哈希算法的设计方法，如果存在冲突的情况下，时间复杂度可能会退化。极端情况下，哈希算法大量冲突，时间复杂度就退化为 O(n*m)。

2.3 BM（Boyer-Moore）算法

参考：http://www.cs.jhu.edu/~langmea/resources/lecture_notes/boyer_moore.pdf

我们把模式串和主串的匹配过程，看作模式串在主串中不停地往后滑动。当遇到不匹配的字符时，BF 算法和 RK 算法的做法是，模式串往后滑动一位，然后从模式串的第一个字符开始重新匹配。

如果我们可以一次性往后滑动好几位，那匹配的效率岂不是就提高了？

BM 算法，本质上其实就是在寻找这种规律，当模式串和主串某个字符不匹配的时候，能够跳过一些肯定不会匹配的情况，将模式串往后多滑动几位。

BM 算法构建的规则有两类，**坏字符规则和好后缀规则。**

- 坏字符规则：If we mismatch, use knowledge of the mismatched text character to skip alignments
- 好后缀规则：If we match some characters, use knowledge of the matched characters to skip alignments

2.4 KMP算法

KMP 算法的核心思想，跟上一节讲的 BM 算法非常相近。我们假设主串是 a，模式串是 b。在模式串与主串匹配的过程中，当遇到不可匹配的字符的时候，我们希望找到一些规律，可以将模式串往后多滑动几位，跳过那些肯定不会匹配的情况

2. 多模式匹配

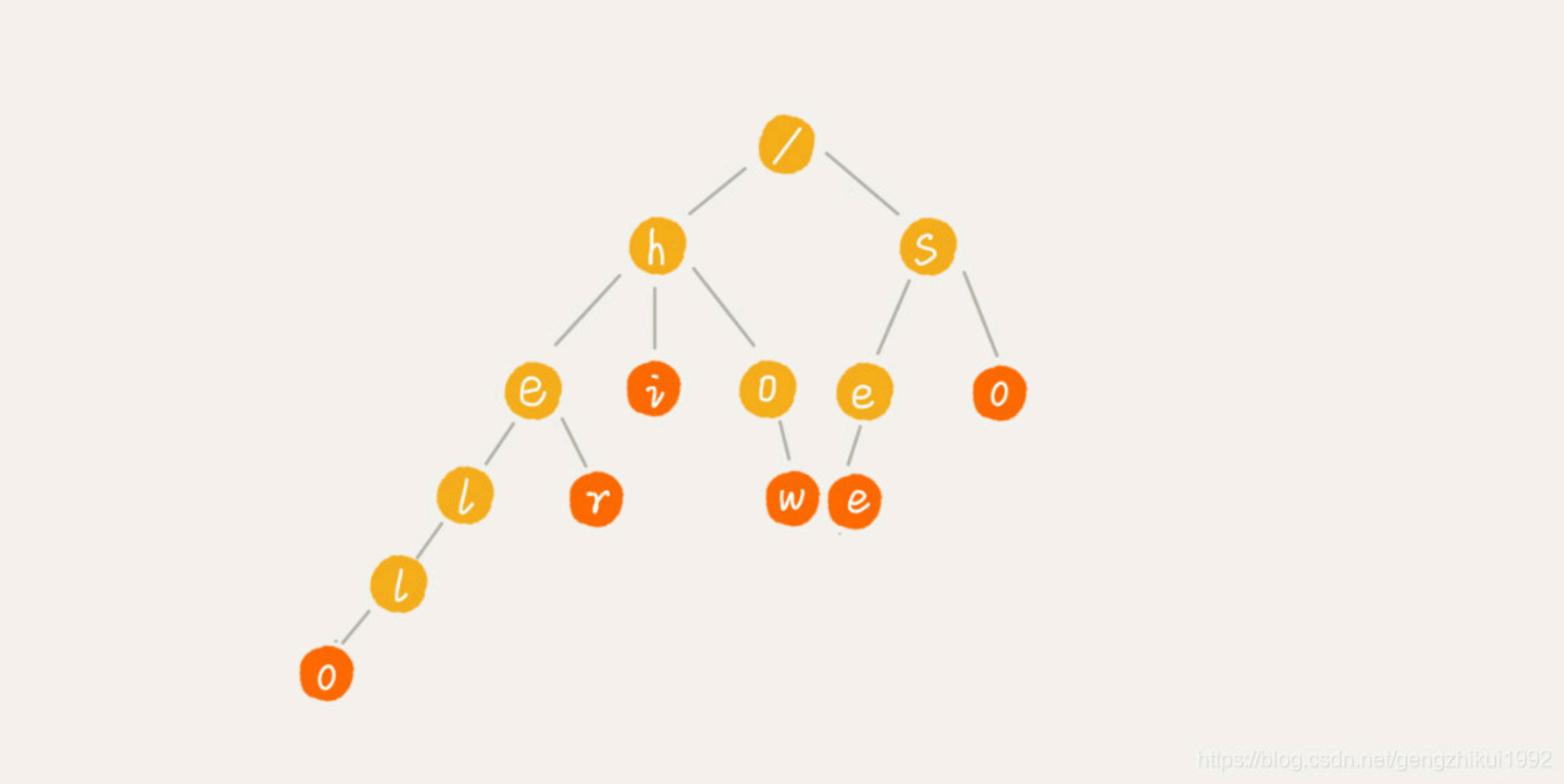
多模式匹配，在一个串中同时查找多个串，常见算法有：AC自动机算法和Trie树算法。

2.1 Trie树

Trie 树，也叫“字典树”。顾名思义，它是一个树形结构。它是一种专门处理字符串匹配的数据结构，用来解决在一组字符串集合中快速查找某个字符串的问题。

举个简单的例子来说明一下。我们有 6 个字符串，它们分别是：how，hi，her，hello，so，see。我们希望在里面多次查找某个字符串是否存在。如果每次查找，都是拿要查找的字符串跟这 6 个字符串依次进行字符串匹配，那效率就比较低，有没有更高效的方法呢？这个时候，我们就可以先对这 6 个字符串做一下预处理，组织成 Trie 树的结构，之后每次查找，都是在 Trie 树中进行匹配查找。

Trie 树的本质，就是利用字符串之间的公共前缀，将重复的前缀合并在一起。



如果要在一组字符串中，频繁地查询某些字符串，用 Trie 树会非常高效。

构建 Trie 树的过程，需要扫描所有的字符串，时间复杂度是 O(n)（n 表示所有字符串的长度和）。

但是一旦构建成功之后，后续的查询操作会非常高效。如果要查询的字符串长度是 k，那我们只需要比对大约 k 个节点，就能完成查询操作。跟原本那组字符串的长度和个数没有任何关系。所以说，构建好 Trie 树后，在其中查找字符串的时间复杂度是 O(k)，k 表示要查找的字符串的长度。

字符串中包含的字符集不能太大，要求字符串的前缀重合比较多，如果字符集太大，那存储空间可能就会浪费很多。即便可以优化，但也要付出牺牲查询、插入效率的代价。

2.2 AC(Aho-Corasick)自动机

假设需要模式串有上万个，通过单模式串匹配算法（比如 KMP 算法），需要扫描几千遍。很显然，这种处理思路比较低效。

Trie 树就是一种多模式串匹配算法。我们用Trie树可以对上千个模式串字典进行预处理，构建成 Trie 树结构。这个预处理的操作只需要做一次，如果字典动态更新了，比如删除、添加了一个模式串，那我们只需要动态更新一下 Trie 树就可以了。

AC 自动机算法，全称是 Aho-Corasick 算法。其实，Trie 树跟 AC 自动机之间的关系，就像单串匹配中朴素的串匹配算法，跟 KMP 算法之间的关系一样，只不过前者针对的是多模式串而已。所以，AC 自动机实际上就是在 Trie 树之上，加了类似 KMP 的 next 数组，只不过此处的 next 数组是构建在树上罢了。

3. Hyperscan

Hyperscan是一款来自于Intel的高性能的正则表达式匹配库。它是基于X86平台以PCRE为原型而开发的，并以BSD许可开源在https://01.org/hyperscan。在支持PCRE的大部分语法的前提下，Hyperscan增加了特定的语法和工作模式来保证其在真实网络场景下的实用性。与此同时，大量高效算法及IntelSIMD*指令的使用实现了Hyperscan的高性能匹配。Hyperscan适用于部署在诸如DPI/IPS/IDS/FW等场景中,目前已经在全球多个客户网络安全方案中得到实际的应用。此外，Hyperscan还支持和开源IDS/IPS产品Snort(https://www.snort.org)和Suricata (https://suricata-ids.org)集成，使其应用更加广泛。

3.1 功能

功能多样

作为纯软件产品，Hyperscan支持Intel处理器多平台的交叉编译，且对操作系统无特殊限定，同时支持虚拟机和容器场景。Hyperscan 实现了对PCRE语法的基本涵盖，对复杂的表达式例如”.”*和*”[^>]”不会有任何支持问题。在此基础上，Hyperscan增加了不同的匹配模式(流模式和块模式)来满足不同的使用场景。通过指定参数，Hyperscan能找到匹配的数据在输入流中的起始和结束位置。更多功能信息请参考http://01org.github.io/hyperscan/dev-reference/。

大规模匹配

根据规则复杂度的不同，Hyperscan能支持几万到几十万的规则的匹配。与传统正则匹配引擎不同，Hyperscan支持多规则的同步匹配。在用户为每条规则指定独有的编号后，Hypercan可以将所有规则编译成一个数据库并在匹配过程中输出所有当前匹配到的规则信息。

流模式（streaming mode）

Hyperscan主要分为两种模式：块模式 (blockmode)和流模式 (streaming mode). 其中块模式为状态正则匹配引擎具有的模式，即对一段现成的完整数据进行匹配，匹配结束即返回结果。流模式是Hyperscan为网络场景下跨报文匹配设计的特殊匹配模式。在真实网络场景下，数据是分散在多报文中。若有数据在尚未到达的报文中时，传统匹配模式将无法适用。在流模式下，Hyperscan可以保存当前数据匹配的状态，并以其作为接收到新数据时的初始匹配状态。如图3所示，不管数据”xxxxabcxxxxxxxxdefx”以怎样的形式被分散在以时间顺序到达的报块中，流模式保证了最后匹配结果的一致性。另外，Hyperscan对保存的匹配状态进行了压缩以减少流模式对内存的占用。Hyperscan流模式解决了数据完整性问题，极大地简化用户网络流处理的过程。

3.2 原理

Hyperscan以自动机理论为基础，其工作流程主要分成两个部分：编译期(compiletime)和运行期(run-time)。

编译期

Hyperscan 自带C++编写的正则表达式编译器。如图1所示，它将正则表达式作为输入，针对不同的IA平台，用户定义的模式及特殊语法，经过复杂的图分析及优化过程，生成对应的数据库。另外，生成的数据库可以被序列化后保存在内存中，以供运行期提取使用。

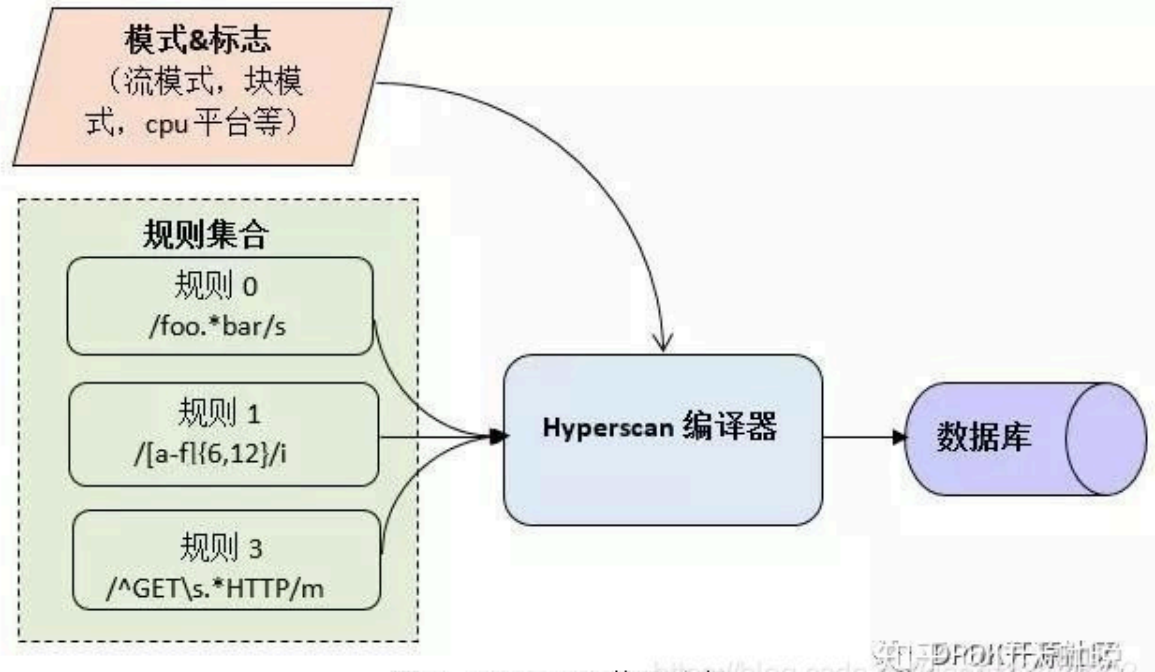


图 1：Hyperscan 编译流程

运行期

Hyperscan的运行期是通过C语言来开发的。

用户需要预先分配一段内存来存储临时匹配状态信息，之后利用编译生成的数据库调用Hyperscan内部的匹配引擎(NFA, DFA等)来对输入进行模式匹配。

Hyperscan在引擎中使用Intel处理器所具有的SIMD指令进行加速。同时，用户可以通过回调函数来自定义匹配发生后采取的行为。由于生成的数据库是只读的，用户可以在多个CPU核或多线程场景下共享数据库来提升匹配扩展性。

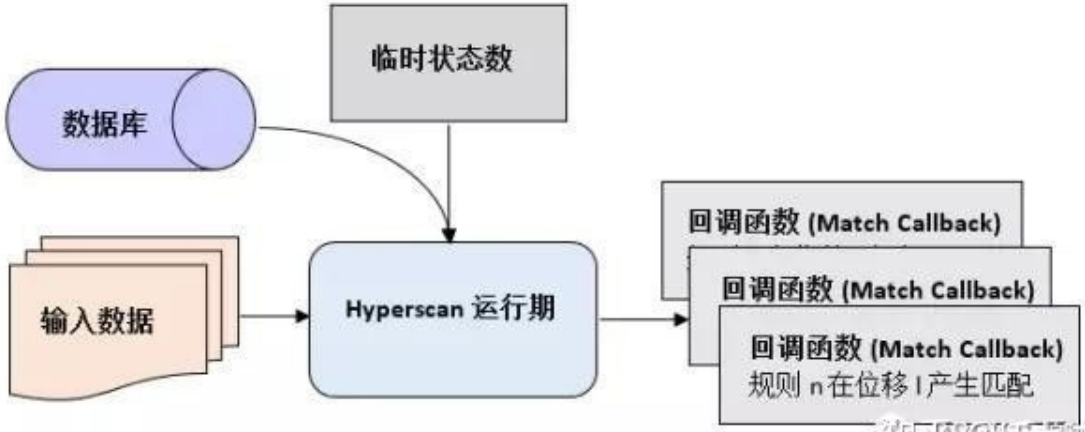


图 2：Hyperscan run-time 流程

https://blog.csdn.net/gengzhikai1992

参考

数据结构与算法之美 王争
Intel高性能正则表达式匹配库——Hyperscan

多模匹配--历程

- 说明
  - AC（Aho-Corasick ）算法
  - WM（Wu-Manber）算法
  - 代码案例说明
  - AC与WM对比：
  - 拓展
  - 代码实现
    - AC算法实现（python实现）
    - WM算法实现（java实现）

说明

多模匹配的一个场景：从一段字符串中匹配多个模式字符串（关键字符串）
多模匹配常应用场景：（1）关键字过滤 （2）入侵检测（3）病毒检测（4）分词等
多模匹配具体算法有很多，常用的有（1）Trie树（2）AC算法（3）WM算法

AC（Aho-Corasick ）算法

1. AC自动机，核心思想是通过有限自动机巧妙地将字符比较转化为了状态转移。;
2. 重要函数： goto函数，failure函数，output函数。;
3. goto函数：用来指导状态转换;;
4. failure函数：指导匹配失败时的状态转换;
5. output函数：描述哪些状态下发生了匹配;

6. 参考地址：
   状态机入门：从定义到使用 http://www.sohu.com/a/230886244_114819
   多模字符串匹配算法-Aho–Corasick–java https://www.cnblogs.com/hwyang/p/6836438.html
   AC自动机+trie树实现高效多模式匹配字典（java源码）https://blog.csdn.net/wangyangzhizhou/article/details/80964748

WM（Wu-Manber）算法

1. 采用哈希的方式以及BM算法中的坏字符规则达到快速检索、跳跃式步进的效果；
2. 重要表结构： SHIFT，HASH，PREFIX表；
3. SHIFT表：坏字符表；
4. HASH表：对应字符块B；
5. PREFIX表：前缀表，用以控制前缀查找，加大效率。
6. 参考地址：（建议先看懂BM算法）
   字符串匹配的Boyer-Moore算法（很详细） http://www.ruanyifeng.com/blog/2013/05/boyer-moore_string_search_algorithm.html
   多模匹配 Wu-Manber算法 https://blog.csdn.net/sunnianzhong/article/details/8875566

代码案例说明

1. 输入：
   关键字数组：["研究", "脂肪酶", "水平下", "哮喘", "正相关", "表达式", "研究结果"]
   待匹配字符："该研究结果表明，IL-17A和IL-9高表达以及脂肪酶、CCL11水平下降与成人哮喘之间呈正相关。"
2. 输出：
   匹配成功关键词组：['研究', '研究结果', '脂肪酶', '水平下', '哮喘', '正相关']
3. 测试说明：使用2万多个关键词，对2000来份字符串进行提取，提取速度均在2秒内

AC与WM对比：

1. 内存占用：WM改进算法比AC改进算法的内存小很多；
2. 预处理： WM改进算法比AC改进算法的预处理时间小很多；
3. 匹配速度：WM算法的匹配速度跟加载的模式串内容有很大的关系；
4. AC算法跟加载的模式串内容无关；
5. 前缀：如果前缀内容大量相似，WM改进算法的Shift表和HASH表冲突比较多，匹配慢。

拓展

1. 模式匹配：数据结构中字符串的一种基本运算，给定一个子串，要求在某个字符串中找出与该子串相同的所有子串，这就是模式匹配。模式匹配分为2类：单模匹配和多模匹配；
2. 单模匹配常见算法：朴素算法（即不断后移一位匹配），Rabin-Karp（朴素算法的基础上进行hash对比），KMP算法（每次匹配后移位数动态计算，增大效率），BM算法（结合坏字符动态移位，从字符末尾匹配）等；
3. AC算法优化：采用双数组来优化内存占用。

代码实现

AC算法实现（python实现）

```python
# -*- encoding=utf-8 -*-


class Node(object):
    def __init__(self):
        self.next = {}
        self.fail = None
        self.isWord = False


class Ahocorasick(object):
    def __init__(self):
        self.__root = Node()

    def addWord(self, word):
        """
            @param word: 添加关键词到Tire树中
        """
        tmp = self.__root
        for i in range(0, len(word)):
            if not tmp.next.__contains__(word[i]):
                tmp.next[word[i]] = Node()
            tmp = tmp.next[word[i]]
```

```
        tmp.isWord = True

    def make(self):
        """
            build the fail function
            构建自动机，失效函数
        """
        tmpQueue = []
        tmpQueue.append(self.__root)
        while (len(tmpQueue) > 0):
            temp = tmpQueue.pop()
            p = None
            for k, v in temp.next.items():
                if temp == self.__root:
                    temp.next[k].fail = self.__root
                else:
                    p = temp.fail
                    while p is not None:
                        if p.next.__contains__(k):
                            temp.next[k].fail = p.next[k]
                            break
                        p = p.fail
                    if p is None:
                        temp.next[k].fail = self.__root
                tmpQueue.append(temp.next[k])

    def search(self, content):
        """
            @return: 返回匹配串的集合，（也可针对需求更改，返回匹配串的位置下标）
        """
        p = self.__root
        result = []
        startWordIndex = 0
        endWordIndex = -1
        currentPosition = 0

```

```
        while currentPosition < len(content):
            word = content[currentPosition]
            # 检索状态机，直到匹配
            while p.next.__contains__(word) == False and p != self.__root:
                p = p.fail

            if p.next.__contains__(word):
                if p == self.__root:
                    # 若当前节点是根且存在转移状态，则说明是匹配词的开头，记录词的起始位置
                    startWordIndex = currentPosition
                # 转移状态机的状态
                p = p.next[word]
            else:
                p = self.__root

            if p.isWord:
                # 若状态为词的结尾，则把词放进结果集
                # result.append((startWordIndex, currentPosition))
                result.append(content[startWordIndex:currentPosition + 1])

            currentPosition += 1
        return result

    def replace(self, content):
        """
            替换部分字符
        """
        replacepos = self.search(content)
        result = content
        for i in replacepos:
            result = result[0:i[0]] + (i[1] - i[0] + 1) * u'*' + content[i[1] + 1:]
        return result


if __name__ == '__main__':
    # 1.创建对象
```

```python
    ah = Ahocorasick()
    # 2.添加关键词（模式串）
    ah.addWord('研究')
    ah.addWord("脂肪酶")
    ah.addWord("水平下")
    ah.addWord("哮喘")
    ah.addWord("正相关")
    ah.addWord("表达式")
    ah.addWord('研究结果')

    # 3.穿件损失函数和输出函数
    ah.make()

    # 4.匹配过程
    mm = ah.search('该研究结果表明，IL-17A和IL-9高表达以及脂肪酶、CCL11水平下降与成人哮喘之间呈正相关。')

    print(mm)
```

WM算法实现（java实现）

```java
import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import java.util.Vector;

/***
 * Wu-Manber多模式关键词快速检测(过滤)算法
 * 说明：详细使用案例详见main方法，建议在关键词超过一定量级时修改加载关键词的方法（内存消耗问题）
 *
 */
public class WuManber {
    private int B=2;//块字符X的长度(模式串后缀字符的个数)

    private boolean initFlag = false;//是否初始化
    private UnionPatternSet unionPatternSet = new UnionPatternSet();
```

```java
    private int maxIndex = (int) java.lang.Math.pow(2, 16); //maxIndex = 65536
    private int shiftTable[] = new int[maxIndex];
    private Vector <AtomicPattern> hashTable[] = new Vector[maxIndex];
    private UnionPatternSet tmpUnionPatternSet = new UnionPatternSet();

    public WuManber() {

    }

    public static void main(String[] args) {
        //1.构建对象
        WuManber objWM=new WuManber();

        //获取待匹配的关键词（模式串）
        Vector<String> vKey=new Vector<>();
        vKey.add("研究");
        vKey.add("哮喘");
        vKey.add("研究结果");
        vKey.add("脂肪酶");
        vKey.add("水平下");
        vKey.add("正相关");
        vKey.add("表达式");


        //2.构建WM算法词典表，确认构建成功
        if(objWM.addFilterKeyWord(vKey, 0)){

           long startTime=System.currentTimeMillis();   //获取开始时间


            String text = "该研究结果表明，IL-17A和IL-9高表达以及脂肪酶、CCL11水平下降与成人哮喘之间呈正相关。";

            //3.进行匹配工作，并对结果进行处理
            List<String> sResult = objWM.macth(text, new Vector(0));

            System.out.println(sResult);
```

```java


            long endTime = System.currentTimeMillis();
            System.out.println("算法运行时间:  "+(endTime-startTime)+"ms");  //算法运行时间
        }

        //4.释放对象存储空间
        objWM.clear();
    }



    /**
     * 匹配工作进行
     * @param content
     * @param levelSet
     * @return
     */
    public List<String> macth(String content, Vector <Integer> levelSet) {
        List<String> sResult = new ArrayList<>();
        if (initFlag == false)
            init();
        Vector <AtomicPattern> aps = new Vector <AtomicPattern>();
        String preContent = content;//preConvert(content);
        for (int i = 0; i  < preContent.length();) {
            char checkChar = preContent.charAt(i);
            if (shiftTable[checkChar] == 0) {
                Vector <AtomicPattern> tmpAps = new Vector <AtomicPattern>();
                tmpAps = findMathAps(preContent.substring(0, i + 1),hashTable[checkChar]);
                aps.addAll(tmpAps);
                if(tmpAps.size()>0){
                    sResult.add(tmpAps.get(0).getPattern().str);
                }
                i++;
            } else
                i = i + shiftTable[checkChar];
```

```
        }
        parseAtomicPatternSet(aps, levelSet);
        return sResult;
    }


    /**
     * 加入关键词，当关键词较多时不推荐使用vector的方式接入参数，因为会严重消耗内存
     * @param keyWord
     * @param level
     * @return
     */
    public boolean addFilterKeyWord(Vector<String> keyWord, int level) {
        if (initFlag == true)
            return false;
        UnionPattern unionPattern = new UnionPattern();
        Object[] strArray = keyWord.toArray();
        for (int i = 0; i  < strArray.length; i++) {
            String sPattern=(String)strArray[i];
            Pattern pattern = new Pattern(sPattern);
            AtomicPattern atomicPattern = new AtomicPattern(pattern);
            unionPattern.addNewAtomicPattrn(atomicPattern);
            unionPattern.setLevel(level);
            atomicPattern.setBelongUnionPattern(unionPattern);
        }
        tmpUnionPatternSet.addNewUnionPattrn(unionPattern);
        return true;
    }

    /**
     * 验证字符
     * @param ch
     * @return
     */
    private boolean isValidChar(char ch) {
        if ((ch >= '0' && ch  <= '9') || (ch >= 'A' && ch  <= 'Z') || (ch >= 'a' && ch  <= 'z'))
```

```
            return true;
        if ((ch >= 0x4e00 && ch  <= 0x7fff) || (ch >= 0x8000 && ch  <= 0x952f))
            return true;// 简体中文汉字编码
        return false;
    }

    /**
     * 封装原子模式集
     * @param aps
     * @param levelSet
     */
    private void parseAtomicPatternSet(Vector <AtomicPattern> aps,Vector <Integer> levelSet) {
        while (aps.size() > 0) {
            AtomicPattern ap = aps.get(0);
            UnionPattern up = ap.belongUnionPattern;
            if (up.isIncludeAllAp(aps) == true) {
                levelSet.add(new Integer(up.getLevel()));
            }
            aps.remove(0);
        }
    }

    /**
     * 查找原子模式
      * @param src
     * @param destAps
     * @return
     */
    private Vector <AtomicPattern> findMathAps(String src,Vector <AtomicPattern> destAps) {
        Vector <AtomicPattern> aps = new Vector <AtomicPattern>();
        for (int i = 0; i  < destAps.size(); i++) {
            AtomicPattern ap = destAps.get(i);
            if (ap.findMatchInString(src) == true)
                aps.add(ap);
        }
        return aps;
```

```
    }

    /**
     * 预转换内容（除掉特殊字符）
     * @param content
     * @return
     */
    private String preConvert(String content) {
        String retStr = new String();
        for (int i = 0; i  < content.length(); i++) {
            char ch = content.charAt(i);
            if (this.isValidChar(ch) == true) {
                retStr = retStr + ch;
            }
        }
        return retStr;
    }

    /**
     * shift table and hash table of initialize
     */
    private void init() {
        initFlag = true;
        for (int i = 0; i  < maxIndex; i++)
            hashTable[i] = new Vector <AtomicPattern>();
        shiftTableInit();
        hashTableInit();
    }

    /**
     * 清除
     */
    public void clear() {
        tmpUnionPatternSet.clear();
        initFlag = false;
    }
```

```java

    /**
     * 初始化跳跃表
     */
    private void shiftTableInit() {
        for (int i = 0; i  < maxIndex; i++)
            shiftTable[i] = B;
        Vector <UnionPattern> upSet = tmpUnionPatternSet.getSet();
        for (int i = 0; i  < upSet.size(); i++) {
            Vector <AtomicPattern> apSet = upSet.get(i).getSet();
            for (int j = 0; j  < apSet.size(); j++) {
                AtomicPattern ap = apSet.get(j);
                Pattern pattern = ap.getPattern();
                //System.out.print(pattern.charAtEnd(1)+"\t");//如pattern.charAtEnd(1)==B,则shiftTable[pattern.charAtEnd(1)]==shiftTable[53]
                if (shiftTable[pattern.charAtEnd(1)] != 0)
                    shiftTable[pattern.charAtEnd(1)] = 1;
                if (shiftTable[pattern.charAtEnd(0)] != 0)
                    shiftTable[pattern.charAtEnd(0)] = 0;
            }
        }
    }

    /**
     * 初始化HASH表
     */
    private void hashTableInit() {
        Vector <UnionPattern> upSet = tmpUnionPatternSet.getSet();
        for (int i = 0; i  < upSet.size(); i++) {
            Vector <AtomicPattern> apSet = upSet.get(i).getSet();
            for (int j = 0; j  < apSet.size(); j++) {
                AtomicPattern ap = apSet.get(j);
                Pattern pattern = ap.getPattern();
                //System.out.println(pattern.charAtEnd(0));//存储shiftTable[pattern.charAtEnd(0)]==0的字符块
                if (pattern.charAtEnd(0) != 0) {
                    hashTable[pattern.charAtEnd(0)].add(ap);
                    //System.out.println(hashTable[pattern.charAtEnd(0)]);
```

```
                }
            }
        }
    }
}

/**
 * 模式类
 */
class Pattern {
    public String str;

    Pattern(String str) {
        this.str = str;
    }

    public char charAtEnd(int index) {
        if (str.length() > index) {
            return str.charAt(str.length() - index - 1);
        } else
            return 0;
    }

    public String getStr() {
        return str;
    }
}

/**
 * 原子模式类
 */
class AtomicPattern {
    private Pattern pattern;
    public UnionPattern belongUnionPattern;
    AtomicPattern(Pattern pattern) {
        this.pattern = pattern;
```

```
    }

    public UnionPattern getBelongUnionPattern() {
        return belongUnionPattern;
    }

    public void setBelongUnionPattern(UnionPattern belongUnionPattern) {
        this.belongUnionPattern = belongUnionPattern;
    }

    public Pattern getPattern() {
        return pattern;
    }

    public void setPattern(Pattern pattern) {
        this.pattern = pattern;
    }

    public boolean findMatchInString(String str) {
        if (this.pattern.str.length() > str.length())
            return false;
        int beginIndex = str.length() - this.pattern.str.length();
        String eqaulLengthStr = str.substring(beginIndex);
        if (this.pattern.str.equalsIgnoreCase(eqaulLengthStr))
            return true;
        return false;
    }
}

/**
 * 合并的模式类
 */
class UnionPattern {
    public Vector <AtomicPattern> apSet;
    private int level;
    // union string
```

```
    UnionPattern() {
        this.apSet = new Vector <AtomicPattern>();
    }
    public void addNewAtomicPattrn(AtomicPattern ap) {
        this.apSet.add(ap);
    }
    public Vector <AtomicPattern> getSet() {
        return apSet;
    }
    public boolean isIncludeAllAp(Vector <AtomicPattern> inAps) {
        if (apSet.size() > inAps.size())
            return false;
        for (int i = 0; i  < apSet.size(); i++) {
            AtomicPattern ap = apSet.get(i);
            if (isInAps(ap, inAps) == false)
                return false;
        }
        return true;
    }
    private boolean isInAps(AtomicPattern ap, Vector <AtomicPattern> inAps) {
        for (int i = 0; i  < inAps.size(); i++) {
            AtomicPattern destAp = inAps.get(i);
            if (ap.getPattern().str.equalsIgnoreCase(destAp.getPattern().str) == true)
                return true;
        }
        return false;
    }
    public void setLevel(int level) {
        this.level = level;
    }
    public int getLevel() {
        return this.level;
    }
}
```

```java
/**
 * 合并的模式集子类
 */
class UnionPatternSet {
    // union string set
    public Vector <UnionPattern> unionPatternSet;
    UnionPatternSet() {
        this.unionPatternSet = new Vector <UnionPattern>();
    }
    public void addNewUnionPattrn(UnionPattern up) {
        this.unionPatternSet.add(up);
    }
    public Vector <UnionPattern> getSet() {
        return unionPatternSet;
    }
    public void clear() {
        unionPatternSet.clear();
    }
}
```